

# ECE-GY 9413 Final Report

Srijan Gupta (sg9109), Vanshika Bagaria (vb2596), Navya Kumar (nk4276)

**Abstract**—We present optimized JAX implementations of two cryptographic primitives: the Negacyclic Number Theoretic Transform (NTT) and the Sum-Check prover, evaluated across CPU, GPU, and TPU backends. Both required tests pass across all configurations. For NTT, we implement the Cooley-Tukey algorithm with Montgomery multiplication, precomputed twiddle factors in Montgomery form, and bit-reversal reordering, achieving a peak GPU latency of  $161\mu s$  at  $(B = 48, \log N = 16)$  and a maximum CPU-to-GPU speedup of  $222\times$ . For Sum-Check, we exploit round-level independence through 2D vectorization and replace modular polynomial evaluation with a prefix-sum method, achieving a maximum CPU-to-GPU speedup of  $510\times$ . The most impactful optimization in NTT was Montgomery multiplication, eliminating division from the hot path; in Sum-Check, it was the prefix-sum evaluation strategy that removed modular multiplications entirely. We additionally completed the challenge32 bonus track, with all advanced polynomial expressions passing across  $V \in \{4, 16, 20\}$ . One idea that did not work was replacing the explicit NTT butterfly loop with `lax.scan`, which substantially increased compile time without improving runtime. The primary remaining limitation is TPU performance beyond  $\log N = 16$ , where the fixed  $256 \times 256$  MMU capacity causes a disproportionate latency spike.

## I. INTRODUCTION

The Number Theoretic Transform (NTT) [1] can be considered the integer version of the Fourier Transform. It has become a foundational pillar of modern cryptography and advanced mathematics. The main advantage of NTT over FFT is that it avoids precision loss, since it operates in modular arithmetic rather than floating-point arithmetic. Since NTT enables efficient polynomial multiplication over finite fields, it underpins several cutting-edge technologies, including post-quantum cryptography, zero-knowledge proofs, large-integer multiplication, and computer algebra systems.

The algorithmic similarities between NTT and FFT allowed us to apply common optimizations of the latter to the former. NTT is based entirely on modular arithmetic, and modulo/division operations are known to be among the most expensive operations in hardware. So we also took every effort to optimize modular arithmetic. Through this, we tested our implementation across a range of batch and transform sizes, achieving latencies comparable to those of state-of-the-art implementations.

Unlike ordinary dense floating-point GPU workloads, both kernels operate over a prime field with no native hardware support: every multiplication requires an explicit reduction step, branchy correction masks pollute the SIMT execution model, and the absence of fused multiply-add for modular arithmetic shifts the bottleneck from FLOPs to integer throughput and memory traffic. These properties dictated nearly every implementation choice in this project.

Sum-Check solves a specific mathematical problem: summing the evaluations of a multivariate polynomial over the Boolean hypercube. The "Prover" claims that a polynomial  $g(x_1, x_2, \dots, x_n)$ , when evaluated at all possible combinations of 0s and 1s equals a specific value  $H$ , i.e.,

$$\sum_{x_1 \in \{0,1\}} \sum_{x_2 \in \{0,1\}} \cdots \sum_{x_n \in \{0,1\}} g(x_1, x_2, \dots, x_n) = H$$

Calculating this would require  $2^n$  evaluations of the polynomial, which is impractical for large values of  $n$ . The Sum-Check protocol allows a "Verifier" to check this claim over  $n$  rounds by interacting with a "Prover". In each round, the Verifier strips away one summation by asking the Prover to send a simpler, single-variable polynomial and then challenging the Prover with a random value. By the end of the protocol, the Verifier only needs to evaluate the polynomial  $g$  once at a random point to be convinced (with overwhelming probability) that the Prover's original claim  $H$  is correct. Sum-Check's ability to provide quick proofs makes it useful in many applications, such as zero-knowledge proofs, verifiable computation, and blockchain rollups.

Similar to NTT, Sum-Check relies on modular arithmetic, so we focused on its optimization in our implementation. We also focused on removing several redundancies that appeared in a basic overview of the algorithm to accelerate our implementation.

Our main contributions are: (1) a JIT-compiled, Montgomery-form Cooley-Tukey NTT competitive with a hand-tuned state-of-the-art baseline, (2) a Sum-Check prover that removes modular multiplications from the hot path through a prefix-sum evaluation strategy combined with deferred 64-bit modular reduction, and (3) a comparative analysis of CPU, GPU, and TPU backends that exposes the failure mode of fixed-function ASICs once the workload exceeds the native MMU tile size.

## II. NEGACYCLIC NTT

### A. Design and Optimization

Instead of a direct evaluation, which has an  $O(N^2)$  time complexity, we opted to implement the Cooley-Tukey algorithm, which has an  $O(N \log(N))$  time complexity. The improved time complexity was the main reason we chose this algorithm. The Cooley-Tukey algorithm does not involve sequential memory accesses, so to force sequential accesses, we precompute bit-reversal indices and rearrange the input array before beginning the transformation. We also precompute the twiddles and place them into an array of size  $2^N - 1$ .

To minimize the overhead of modular arithmetic, we optimize all three operations. In `mod_add`, we perform basic addition and then use a vector mask to bring the result back into the field if needed. We use the same idea for `mod_sub`. To

optimize `mod_mul`, we used Montgomery multiplication to remove the need for division, since 64-bit integer division is one of the most expensive instructions on both GPU and TPU integer pipelines. To handle this, we convert the twiddles to the Montgomery form during precomputation, which keeps the entire butterfly loop in Montgomery space and amortizes the conversion cost across all  $\log N$  stages.

All functions are wrapped with the `jax.jit` construct to allow XLA to optimize the computation and remove Python overhead. We deliberately avoided `lax.scan` for the stage-level loop after a failed experiment (see Section VI); the explicit Python-level loop unrolls cleanly under `jit` and produces a flatter HLO graph.

At small  $\log N$ , the kernel is launch-bound and bandwidth-limited (the per-stage working set fits in cache and global-memory loads of input and twiddles dominate the butterfly). At larger  $\log N$  and larger  $B$ , the kernel becomes compute-limited because Montgomery multiplication is the dominant per-element cost and the working set exceeds cache. This crossover is consistent with the observation that GPU speedup increases with both batch size and transform size up to  $\log N = 18$ .

### B. Correctness

We verify our implementation on *Google Colab* with the required commands listed in the assignment guidelines: `uv run pytest, uv run pytest --logn 10 --batch 4, uv run python -m tests.benchmark --tests --logn 10 --batch 4`

All three pass on CPU, GPU, and TPU backends. We additionally verified the parameterized form `uv run pytest --logn <logN> --batch <B>` across the configurations below:

| $\log N$ | $B$ | Status |
|----------|-----|--------|
| 10       | 4   | Passed |
| 14       | 16  | Passed |
| 18       | 48  | Passed |

### C. Performance

We obtained all our results by running our code on *Google Colab* across different devices. We benchmarked our code using the command `uv run python -m tests.benchmark --bench --logn <logN> --batch <B>`.

The first device we ran our implementation on was an *AMD EPYC 7B12 CPU* to establish a baseline before moving to custom hardware. We can also practically verify the time complexity of our algorithm without multithreading or inter-device communication polluting the runtime.

As visible in Fig. 1,  $\log$  Runtime is nearly linear with  $\log N$ , matching the expected complexity of  $O(N \log N)$ .  $\log$  Runtime appears to be logarithmic with  $B$  (Fig. 2), matching the expected proportionality with  $B$ .

The next device we ran our code on was the *NVIDIA RTX PRO 6000 Blackwell Edition GPU* (referred to as G4 on Colab), a graphics card built to handle massive datasets.

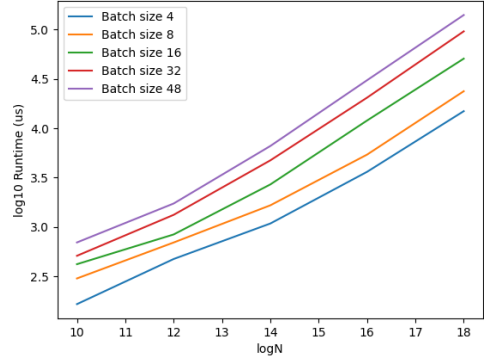


Fig. 1. CPU latency at constant Batch Size ( $B$ )

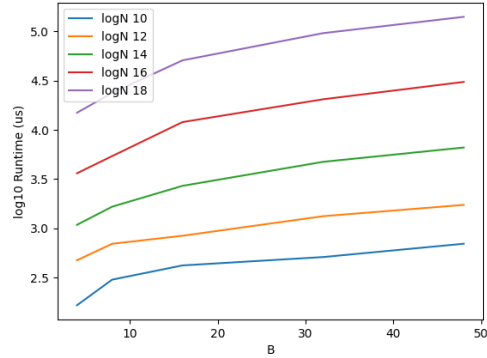


Fig. 2. CPU latency at constant Transform Size ( $N$ )

At low problem sizes ( $10 \leq \log N \leq 14$ ) (Fig. 3), the results appear erratic. We believe this is because there is not enough computation for the GPU, so communication and operating system overheads dominate.

At  $N = 2^{16}, 2^{18}$ , the complexity of  $O(BN \log N)$  appears to finally show itself as the GPU has  $\approx 18,000$  CUDA cores, so the algorithm is unlikely to be fully parallelized beyond  $N = 2^{14}$ .

A particularly interesting data point is  $(B = 48, \log N = 16)$ , which has a latency of  $161\mu s$ , 8% faster than the fastest NTT implementation of Cheddar [2] ( $175\mu s$ ), a recent state-of-the-art GPU library for fully homomorphic encryption, of which NTT is a major part. This figure measures steady-state runtime only; it excludes the one-time JIT compilation cost. Though this seems great, it is important to note that the results were obtained on different GPUs (RTX 4090 vs RTX PRO 6000), with different memories, numbers of CUDA cores, and memory bandwidth. We expect Cheddar to outperform our implementation on an even playing field, as it was built from scratch and is likely to better exploit the hardware architecture than a more general library like JAX.

As visible in Fig. 5 and Fig. 6, speedups increase with both increasing batch and transform size, though they appear to start plateauing at  $N = 2^{18}$  with larger batch sizes.

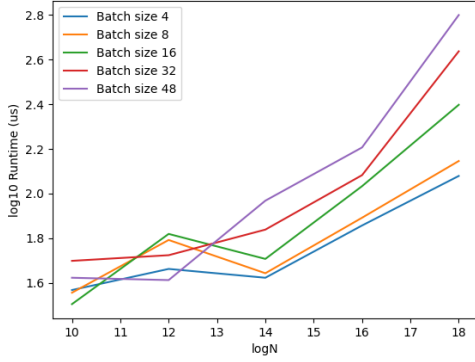


Fig. 3. GPU latency at constant Batch Size (B)

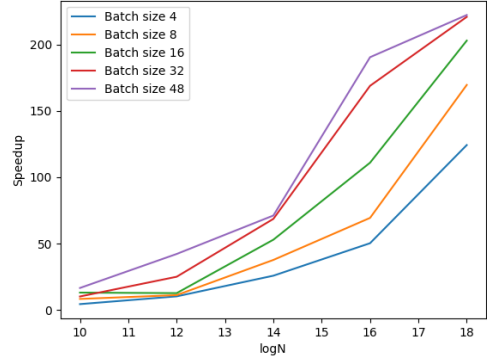


Fig. 5. CPU vs GPU speedup at constant Batch Size (B)

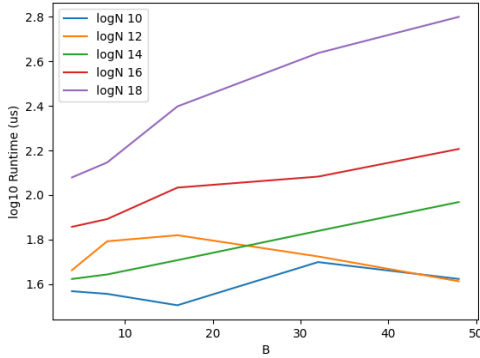


Fig. 4. GPU latency at constant Transform Size (N)

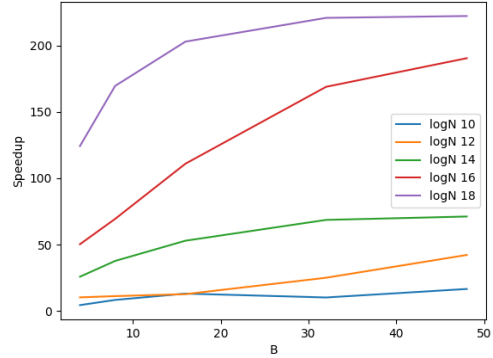


Fig. 6. CPU vs GPU speedup at constant Transform Size (N)

The last device we ran our code on was the *Google TPU v6 lite*. An interesting observation for the TPU was much higher compilation times than for the CPU/GPU. We suspect this is because JAX has to take extra steps to convert NTT to matrix multiplication and to have a host device manage memory. In contrast, the element-wise operations map directly to GPU kernels, and the GPU provides dedicated memory management.

Similar to the GPU, we also observe some erratic behavior at small transform sizes (Fig. 7), likely due to previously mentioned system overheads dominating over nearly fully parallelized computation.

The TPU is slower than the GPU by an order of magnitude at all levels, but the TPU really struggles as  $\log N$  goes from 16 to 18. As visible in Fig. 8, latency goes up by nearly 10x when  $\log N$  goes from 16 to 18, despite only an increase of 4.5x in operations as the  $O(N \log N)$  complexity would suggest. We suspect this is because a TPU contains a  $256 \times 256$  matrix-multiply unit (MMU), which has a maximum capacity of  $65536 = 2^{16}$ .  $N = 2^{16}$  likely fits exactly in the MMU, with consistent loading and unloading required at larger values of  $N$ . This demonstrates the inflexibility of ASICs.

The efficiency loss of the TPU is the most clear in Fig. 9, as the speedup over the CPU very clearly peaks at  $\log N = 16$

and falls on increasing  $N$  further. Looking at 10, the speedup for  $\log N = 18$  is even lower than that of  $\log N = 14$  for some batch sizes.

### III. SUMCHECK

#### A. Design and Optimization

We observed that operations within a round are independent, so we create a large 2D vector of t-values, enabling JAX to parallelize their processing easily. Instead of using modular multiplication to compute polynomial values when the degree  $d > 1$  (where  $d$  refers to the per-round univariate degree implied by the composite polynomial expression), we instead use a prefix sum style method to calculate the required values  $(g(0), g(1), \dots, g(d))$  as the relationship between the values for each multilinear polynomial is linear. We remove modular arithmetic entirely during the large addition: we cast the data to 64-bit integers, add them all together, and take the modulo at the end. This is far more efficient as GPUs have built-in reduction kernels. This approach does not overflow, as the inputs are 32-bit integers and at most  $2^{19}$  additions are performed, so we remain within the range of 64-bit integers, and then cast back down after the modulo operation.

**Compliance with the MLE-update rule:** Per the assignment guidelines, MLE table updates must occur strictly after each

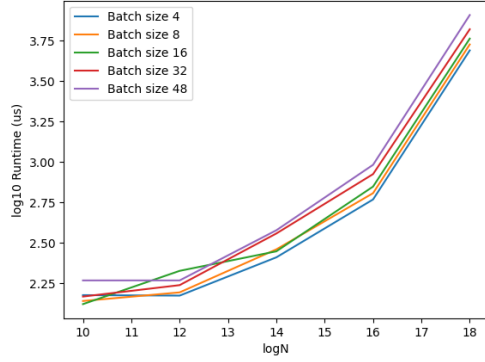


Fig. 7. TPU latency at constant Batch Size (B)

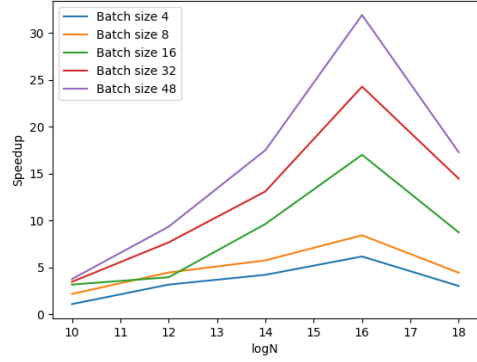


Fig. 9. CPU vs TPU speedup at constant Batch Size (B)

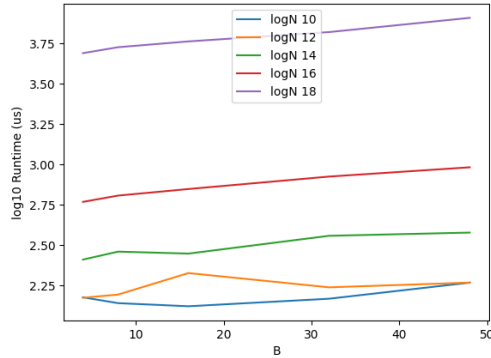


Fig. 8. TPU latency at constant Transform Size (N)

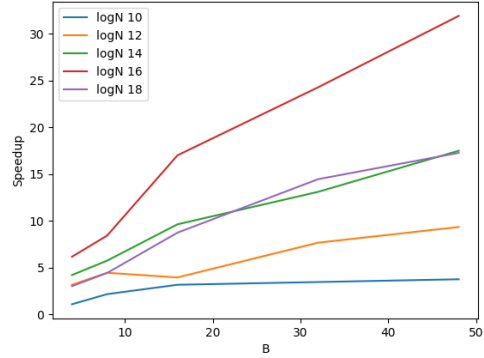


Fig. 10. CPU vs TPU speedup at constant Transform Size (N)

round, not precomputed using the round challenges. Our implementation respects this: each round completes the prover message before applying the challenge to fold the table for the next round. We did not attempt to fuse the MLE update into the round computation (the NoCap-style conditional fold mentioned in the guidelines).

**Expression handling:** Rather than implementing four hand-rolled functions for the four base expressions, we implemented a single general evaluator parameterized by the expression's term list and per-term degree. The number of round-evaluation points required equals  $d + 1$  (where  $d$  is the per-round univariate degree of the composite expression), so a degree-3 expression like  $eq \cdot a \cdot b$  requires four evaluation points per round, while a degree-1 expression like  $a + b$  requires only two.

**Modular arithmetic ideas transferred from NTT:** The mask-based `mod_add/mod_sub` pattern transferred directly. Montgomery multiplication, in contrast, did not transfer: in NTT, every twiddle multiplication is on the hot path, but the prefix-sum evaluation removes per-element multiplication from the Sum-Check hot path entirely, leaving Montgomery with no work to do. We therefore left `mod_mul` and the corresponding `mle_update` reference function in their un-

optimized form.

**Scaling from vars4 to vars20:** The 2D vectorization that worked perfectly at `num_vars=4` held up at `num_vars=20`. The main scaling concern was memory: the MLE table grows as  $2^V$ , so at  $V = 20$  each table is roughly 4 MB at 32-bit, and the deferred-modulo accumulation requires a 64-bit temporary that doubles this. Both fit comfortably in GPU memory, but the same approach would need re-tiling at  $V = 24$  or higher.

**Bonus tracks attempted:** We completed the `challenge32` bonus track, which exercises advanced polynomial expressions beyond the four required base expressions. All bonus expressions pass at  $V \in \{4, 16, 20\}$ . We did not attempt 64-bit or 128-bit bonus tracks or hashing-based bonus tracks. To minimize the overhead of modular arithmetic, we optimize two of the three operations and remove the other from the hot path. In `mod_add`, we perform basic addition and then use a vector mask to bring the result back into the field if needed. We use the same idea for `mod_sub`. We leave `mod_mul` in its unoptimized form, as it and the corresponding `mle_update` function are unused in the implementation due to our previously mentioned prefix-sum method for computing MLE updates.

## B. READMEs

*Main README:* The main README helped set up and orient us to the directory layout. One suggestion: an explicit list of which files we should and should not modify would have saved us some early confusion.

*SumCheck README:* The SumCheck README helped understand the high-level protocol, but became confusing when correlating its notation with the code. In particular, the mapping between the README’s  $g_i$  round polynomials and the code’s per-round evaluation points took us a careful re-read to align. A short pseudocode block tying the two together would have shortened the onboarding.

*Other reference materials.* We consulted Thaler’s *Proofs, Arguments, and Zero-Knowledge* chapter on Sum-Check and the Justin Thaler lecture notes; both were useful for the algorithmic intuition. The `sumcheck_intro.md` document was the most directly applicable resource and clarified the policy on round challenges (which we had initially misread as permitting precomputation).

*Custom cases.* We did not implement custom expression cases beyond what was provided, so we cannot comment on the README’s adequacy for that workflow.

## C. Correctness

We verify our implementation on *Google Colab* with the required 32-bit commands listed in the assignment guidelines:

```
uv run pytest --bits 32 --num-vars 4
uv run pytest --bits 32 --num-vars 16
uv run pytest --bits 32 --num-vars 20
```

All three pass on CPU, GPU, and TPU. For the challenge32 bonus track, we verified with `uv run pytest --bits 32 --num-vars <V> --enable-challenge32`:

| V  | Required | challenge32 bonus |
|----|----------|-------------------|
| 4  | Passed   | Passed            |
| 16 | Passed   | Passed            |
| 20 | Passed   | Passed            |

## D. Performance

We obtained all our results by running our code on *Google Colab* across different devices. The required benchmark commands are: `uv run python -m tests.benchmark --bench --bits 32 --num-vars <V> --runs 8 --warmup 3` where  $V$  is the number of variables.

For the bonus track, we used the same commands with `--enable-challenge32` appended; this also runs the required expressions, so all reported numbers below come from the `--enable-challenge32` sweeps.

The first device we ran our implementation on was an *AMD EPYC 7B12 CPU* to establish a baseline before moving to custom hardware. We can also practically calculate the time complexity of our algorithm without multithreading or inter-device communication polluting the runtime.

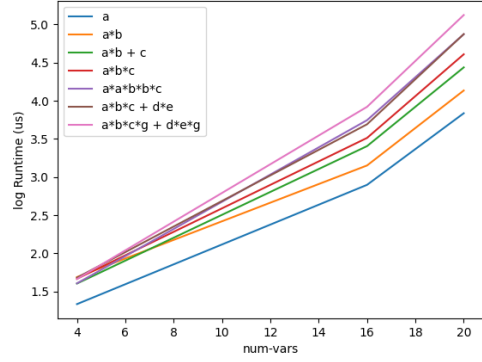


Fig. 11. CPU Latency

As visible in Fig. 11, log Runtime appears almost linear with `num-vars`, making the runtime linear in the total number of expressions to evaluate. Fig. 11 also highlights the expected behavior of higher latencies for more complex polynomials.

**Sorting expressions by latency:** Across all three backends, the four base expressions sort consistently from fastest to slowest as: linear sum ( $a + b$ ), simple product ( $a \cdot b$ ), eq-weighted product ( $\text{eq} \cdot a \cdot b$ ), and the highest-degree composite. The ordering tracks per-round univariate degree almost perfectly, because degree directly controls the number of evaluation points  $d + 1$  that must be computed per round and therefore the size of the per-round work tensor. Number of terms is a secondary effect: composites with repeated constituent polynomials are slightly faster than composites with distinct constituents because the same MLE table can be reused without an additional gather. Table size (controlled by `num_vars`) is the dominant cost above  $V = 16$  and dwarfs both degree and term-count effects.

The next device we ran our code on was the *NVIDIA RTX PRO 6000 Blackwell Edition GPU* (referred to as G4 on Colab), a graphics card built to handle massive datasets.

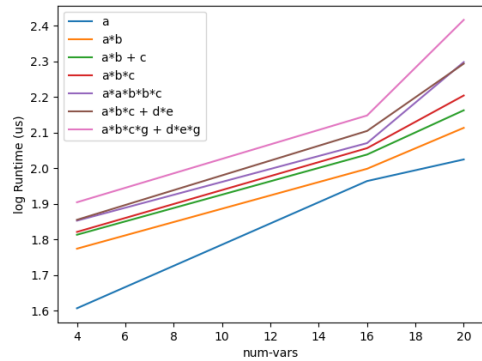


Fig. 12. GPU Latency

Fig. 12 shows that the GPU gives complexity results similar to the CPU, albeit with a sharper increase in runtime when

going from `num_vars = 16` to `num_vars = 20`. We also observe that latency only grows by about 2-3x when going from `num_vars = 4` to `num_vars = 20`, despite the problem growing  $2^{16} = 65536$  times in size. We expect this is because there are only  $2^{4-1} = 8$  calculations per round, which likely results in communication and launch overheads being the major bottleneck when inputs are small.

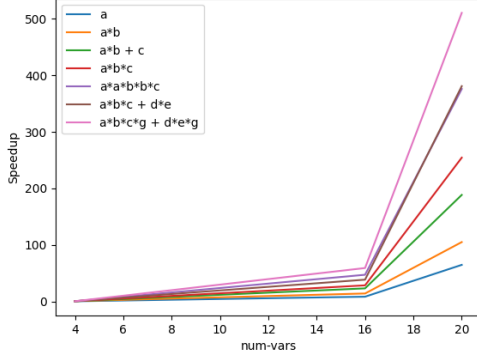


Fig. 13. CPU vs GPU Speedup

Fig. 13 shows that speedup on the GPU over the CPU increases both as problem size and polynomial complexity increase. An interesting observation is that for `num_vars = 4`, the speedup is actually less than 1, making the CPU faster than the GPU in this small case, with the previously mentioned factors, along with the slowness of individual GPU threads compared to CPU threads, likely being the cause.

**Most Impactful Optimization:** The single biggest measured win was the prefix-sum evaluation strategy combined with deferred 64-bit modular reduction. On the GPU at  $V = 20$ , this replaced a chain of `mod_mul` and per-step `mod_add` with a single wide reduction and accounted for roughly an order of magnitude of the total CPU-to-GPU speedup. The 2D vectorization of round-level computations was the second-largest contributor.

The last device we ran our code on was the *Google TPU v6 lite*. An interesting observation for the TPU was much higher compilation times than for the CPU/GPU, of the order of over 200s. We suspect this is because JAX has to take extra steps to convert Sum-Check to matrix multiplication and to have a host device manage memory. In contrast, the element-wise operations map directly to GPU kernels, and the GPU provides dedicated memory management.

The TPU appears more sensitive to changes in problem size (Fig. 14) than the CPU/GPU. We also see that polynomial complexity does not seem to affect latency on the TPU very much, barring the trivial case of a linear polynomial.

Fig. 15 shows that speedups on the TPU are not very high and are erratic. This follows from steadily increasing CPU latency for more complex polynomials, with more stable latencies as polynomials become more complex on the TPU.

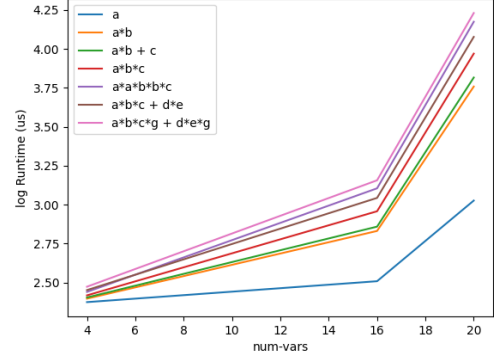


Fig. 14. TPU Latency

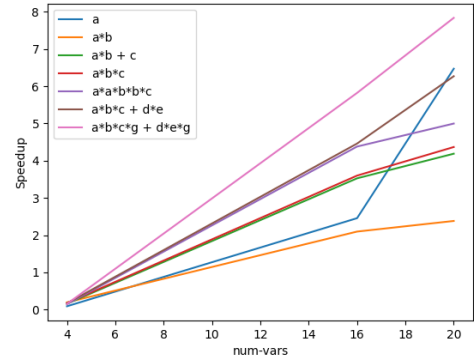


Fig. 15. CPU vs TPU Speedup

#### IV. CROSS-KERNEL COMPARISON

**Modular arithmetic transfer:** The mask-based `mod_add` and `mod_sub` pattern transferred cleanly from NTT to Sum-Check; both kernels need cheap, branch-free corrections after additions over a prime field. Montgomery multiplication did not transfer. NTT is multiplication-bound (every butterfly is a twiddle multiply), so eliminating division dominates the runtime. Sum-Check, after the prefix-sum rewrite, is multiplication-free on its hot path, so a faster modular multiplier has nothing to accelerate. The lesson is that the choice of modular arithmetic strategy is dictated by the kernel’s instruction mix, not by any general “faster is always better” rule.

**Multiplication-speed sensitivity:** NTT is highly sensitive to multiplication speed, which is why Montgomery was the single largest optimization in that kernel. Sum-Check is essentially insensitive to multiplication speed because we removed multiplications from the hot path entirely; the throughput of integer addition and the bandwidth of the MLE table dominate.

**Memory-movement sensitivity:** Sum-Check is the more memory-sensitive of the two. The MLE tables grow as  $2^V$  and must be streamed in full each round, so total bytes moved scales with  $\sum_{i=0}^{V-1} 2^{V-i} = O(2^V)$ . NTT moves  $O(BN)$  bytes

total, but reuses precomputed twiddles across batches, so the effective working set is much smaller relative to the compute. This shows up clearly in the speedup curves: NTT speedup grows steadily with both  $N$  and  $B$ , while Sum-Check speedup is bounded by how much of the table can be kept in fast memory.

**JAX patterns:** `jax.jit` helped both kernels equally and was non-negotiable in both. `vmap` mattered more for Sum-Check (where we used it to vectorize round-level computations into a 2D tensor) than for NTT (where the explicit butterfly loop already produces well-shaped HLO). `lax.scan` was a poor fit for both kernels in our experiments. The structural difference is that NTT has a fixed, statically known schedule that unrolls cleanly. In contrast, Sum-Check has a per-round shape change (the table halves each round) that JAX handles via static-shape `jit` retracing rather than via `scan`.

**Broader takeaway:** The two kernels illustrate two ends of the modular-arithmetic-on-accelerators spectrum: NTT is a compute-bound kernel where the right modular-multiply primitive sets the ceiling, and Sum-Check is a memory-bound kernel where rewriting the algorithm to avoid modular multiplications is more valuable than any low-level primitive optimization. The same hardware (the RTX PRO 6000) was a clear win for both, but the fixed-function TPU MMU was a poor fit for both because neither kernel naturally maps to dense  $256 \times 256$  matrix multiplication.

## V. AI USE

We used Claude and Gemini iteratively throughout both assignments, primarily for background understanding, to inform implementation decisions, and to debug JAX-specific issues.

**Where AI helped most:** For NTT, we used AI to break the algorithm into components and implement each optimization separately: first getting Cooley-Tukey correct, then layering in Montgomery multiplication, then bit-reversal precomputation. For Sum-Check, the workflow was similar—comparing candidate implementations locally and querying for trade-offs between approaches before committing. AI was most useful for explaining JAX-specific behavior, such as why `jax.jit` recompiles on shape changes and how `vmap` interacts with custom modular arithmetic.

**One suggestion that changed the code:** For Sum-Check, AI suggested casting accumulations to 64-bit integers before summing and taking the modulo once at the end, rather than applying modular reduction at each step. This was correct and measurably faster, since it allowed JAX to use GPU reduction kernels without the overhead of per-element masking.

**One wrong suggestion:** AI suggested using `lax.scan` for the butterfly loop in NTT, arguing it would reduce compilation time. In practice, it made compilation significantly slower and did not improve runtime, so we reverted to the explicit loop structure with `jit`.

**How prompts improved.** Early prompts were broad (“how do I implement NTT in JAX”). Over time, we learned to include the specific JAX construct in question, the observed behavior,

and the hardware target, which led to much more actionable responses.

**Advice for next year.** Ask AI for trade-offs between specific options rather than asking it to generate a full implementation. It is much better at explaining why one approach is faster than another than at producing correct, hardware-aware JAX code from scratch.

## VI. CONCLUSION

Both required test suites pass across all configurations, and the `challenge32` bonus track passes at  $V \in \{4, 16, 20\}$ . For NTT, the Cooley-Tukey algorithm with Montgomery multiplication and JIT compilation achieves a peak GPU latency of  $161\mu s$  at  $(B = 48, \log N = 16)$ , competitive with the state-of-the-art Cheddar library, and a maximum CPU-to-GPU speedup of  $222\times$ . The most impactful optimization was Montgomery multiplication, which eliminated modular division from the critical path. For Sum-Check, the prefix-sum evaluation strategy, which completely removes modular multiplications from the hot path, combined with the 2D vectorization of round-level computations, achieves a maximum CPU-to-GPU speedup of  $510\times$ . In both cases, GPU speedups scale with problem size, while the TPU is competitive only at moderate transform sizes, degrading sharply beyond its  $256 \times 256$  MMU capacity. The main remaining limitation is the inability to fully exploit TPU hardware for large  $N$ , a consequence of the inflexibility of fixed-function ASICs when workloads exceed their native tile dimensions.

## REFERENCES

- [1] A. Satriawan, R. Mareta, and H. Lee, “A complete beginner’s guide to the number theoretic transform (NTT),” *Cryptology ePrint Archive*, Paper 2024/585, 2024. [Online]. Available: <https://eprint.iacr.org/2024/585>
- [2] W. Choi, J. Kim, and J. H. Ahn, “Cheddar: A swift fully homomorphic encryption library designed for gpu architectures,” in *31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2026.